# CDT204 – Computer Architecture

## Date: Jan. 9th 2025

## Time: 8:30 – 12:30

## Help: Small calculator is allowed in the exam

The exam has 46 points. And 4 BONUS points. The grades will be awarded as follows:

- 3 : 24 Points
- 4 : 33 Points
- 5 : 42 Points

**Important Notes:**

- Give as full an answer as possible to obtain full marks. All calculations, approximations, assumptions and justifications must be reported for full credit unless stated otherwise. Please use figures and examples to clarify.
- If you do not understand a question clearly, you are allowed to call the teacher and ask.
- Write the question and part number on each page clearly.
- Answer each question on a separate page.
- In case you might have forgotten:

  $1GB = 2^{10}MB = 2^{20}KB = 2^{30}B$

  $1\ sec = 10^{3}\ ms = 10^{6}\ \mu s = 10^{9}\ ns = 10^{12}\ ps$

*Live long and prosper*

## *Task 1 - General (3p)*

Consider the following binary representation: 1101 1001

- What is the hexadecimal representation of that number? (1p)
- What is its value in decimal form (base 10) if it contains an unsigned integer? (1p)
- What is its value in decimal form (base 10) if it contains a two's compliment signed integer? (1p)

## *Task 2 - Performance (6p)*

When a program is adapted to run on multiple processors in a multiprocessor system, the execution time on each processor consists of computing time and the overhead time required for locked critical sections and/or to send data from one processor to another.

Assume that you have a 6-core chip that you want to use to solve a given problem:

- You can use as few as 1 core or as many as 6 cores to solve the problem.
- The problem requires 450,000 iterations of a main loop to complete.
- Each loop iteration requires 100 clock cycles.
- Any startup overhead can be ignored (i.e. instructions outside of the loop to instantiate variables, etc.).

If more than 1 core is used to solve a problem, the overhead must be added to the total execution time. This overhead is a function of number of cores and can be defined as: $t_{overhead}=10(X-1)$ where $X$ is the number of processors used and $t_{overhead}$ is the number of cycles spent in overhead code per iteration of the loop. For example total overhead for using 2 cores will be: 10*(2-1)*450000=450000 cycles.

- How many cores should be used to solve this problem the fastest? (4p)
- What is the speedup for it? (2p)

## *Task 3 – MIPS Assembly (4p)*

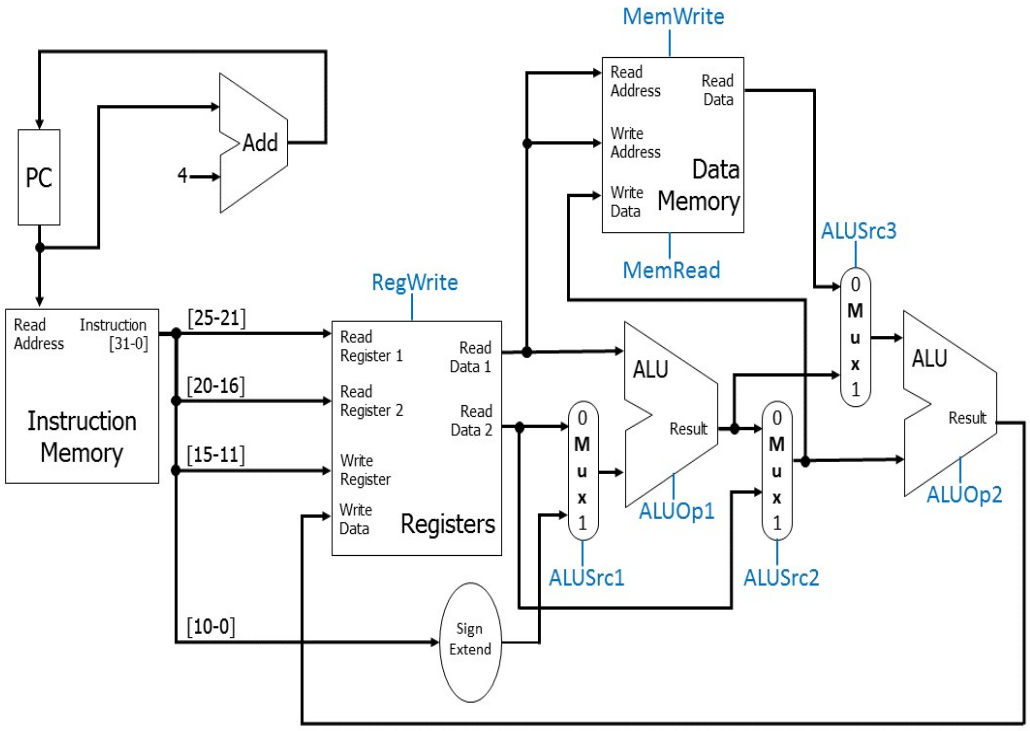
For the following MIPS assembly instructions below:

```
sll     $t0, $s0, 2
add     $t0, $s6, $t0
sll     $t1, $s1, 2
add     $t1, $s7, $t1
lw      $s0, 0($t0)
addi    $t2, $t0, 4
lw      $t0, 0($t2)
add     $t0, $t0, $s0
sw      $t0, 0($t1)
        ...
```

- What is the corresponding C statement? Assume that the variables *f*, *g*, *h*, *i* and *j* are assigned to registers *$s0, $s1, $s2, $s3*, and *$s4*, respectively. Assume that the base address of the arrays *A* and *B* are in the registers *$s6* and *$s7*, respectively. (3p)
- If the variables have the following values before the code runs, what will be their values after execution the code? *f=1, g=2, h=0, i=0, j=1, A={32, 10, 7}, B={99,99,12}* (1p)

**NOTE**: The whole MIPS code translates to only one C statement, not a C program. So the final result for "part a" should be just a C statement, although you must show step-by-step how reached to that conclusion.

## *Task 4 – Datapath (12p)*




The datapath above supports the following instructions:

- **`lw_add rd, (rs), rt`** (fused load and add): Register `rd` will be set to the summation of memory value addressed by register `rs` and the value stored in register `rt`, this can be represented as: `R[rd] = Mem[R[rs]]+R[rt]`
- **`addi_st (rs), rs, imm`** (fused immediate add and store): The memory addressed by register `rs` will be set to summation of register `rs` and constant value `imm`, this can be represented as: `Mem[R[rs]]=R[rs]+imm`
- **`sll_add rd, rs, rt, imm`** (fused immediate left shift and sum): Value of register `rs` is shifted left by constant value `imm` then added to register `rt` and stored in register `rd`, this can be represented as: `R[rd]=(R[rs]<<imm)+R[rt]`

All instructions use the same format (shown below). Not all instructions use all the fields.

| **op** (bits 31-26) | **rs** (bits 25-21) | **rt** (bits 20-16) | **rd** (bits 15-11) | **imm** (bits 10-0) |
|---|---|---|---|---|

For each of the above instructions, specify how the control signals should be set for correct operation. Use X for don't care. ALUOp can be ADD, SUB, SLL, PASS_A, or PASS_B (i.e. PASS_A means pass the top operand

through without changing it).

You can find an empty table for this task at the last page of the exam sheet, which you can use to write your answer in. Detach it from the exam sheet and put it in your answers folder.

## *Task 5 – Virtual Memory (7p)*

Suppose that you have a page size of 4 KB. Also suppose that you have an application in which the text segment (program code) takes up 2 KB, the data segment (global variables, etc.) takes up 5 KB, and that 1 KB allocated for the stack.

- We do not know where in the process's private memory space these three segments are allocated, but we know that they are in 3 distinct address ranges. What is the maximum number of pages that this program might need? (1p)

Now assume that we know that the code segment is allocated in address $1000_{16}$ to $17FF_{16}$, data segment from $10000_{16}$ to $113FF_{16}$ and the stack from $FFC00_{16}$ to $FFFFF_{16}$ in a 1MB virtual memory. After the program runs for a while, some parts of the page table look like the following:

| Valid | Dirty | Virtual Page No. | Physical Page No. |
|---|---|---|---|
| 1 | 0 | 00010000 | 001100101 |
| 1 | 0 | 00000001 | 000110011 |
| 1 | 1 | 11111111 | 110110100 |
| 0 | 0 | 00000001 | 111001000 |
| 1 | 1 | 00010001 | 000110101 |

- Considering that the virtual memory is 1MB in size, how many bits the virtual addresses should be? (1p)

- Judging by the physical page numbers, how big is the physical memory? (1p)

- What page number the code segment has got? (The whole segment is placed in one page) (2p)

- Is all of the data segment read into the physical memory? (2p)

## *Task 6 – Cache Performance (5p)*

The average memory access time for a processor developed by Tyrell Corporation with 1 level of cache is 2.4 clock cycles. The properties of the memory system are as follows:

- If data is present in the cache and is valid, it can be accessed in 1 clock cycle.
- If data is not found in the cache, 80 clock cycles are needed to get it from the memory.

As a designer who works for Tyrell Co. you are given the task of improving average memory access times by 65%. To achieve that goal you are considering adding a 2nd level of cache. This new cache has the following properties:

- The new L2 cache can be accessed in 6 clock cycles.
- The addition of this new cache does not affect the L1 cache's access patterns or hit times.
- Accessing the main memory would still require 80 additional clock cycles.

To obtain the desired improvement and keep your boss happy, how often must data be found in L2 cache? (in other words, what should be the hit-rate of L2 cache?)

## *Task 7 – Multiprocessing (9p)*

Serena Kogan is a programmer at Cyberdyne Systems, and she has been asked to parallelize an old program so that it runs faster on modern multicore processors.

- She parallelizes the program and discovers that its speedup over the single-threaded version of the same program is significantly less than the number of processors. She finds that many cache invalidations are occurring in each core's data cache. What program behavior could be causing these invalidations? Explain. (1p)
- She modifies the program to fix this first performance issue. However, now she finds that the program is slowed down by a global state update that must happen in only a single thread after every parallel computation. In particular, the program performs 90% of its work (measured as processor-seconds) in the parallel portion and 10% of its work in this serial portion. The parallel portion is perfectly parallelizable. What is the maximum speedup of the program if the multicore processor had an infinite number of cores?(3p)
- How many processors would be required to attain a speedup of 4?(2p)

In order to execute the program with parallel and serial portions more efficiently, Cyberdyne decides to design a custom heterogeneous processor with following properties:

- This processor will have one large core (which executes code more quickly but also takes greater

die area on-chip) and multiple small cores (which execute code more slowly but also consume less area), all sharing one processor die.

- When the program is in its parallel portion, all of its threads execute **only** on small cores.
- When the program is in its serial portion, the one active thread executes on the large core.
- Performance (execution speed) of a core is proportional to the square root of its area.
- Assume that there are 16 units of die area available. A small core must take 1 unit of die area. The large core may take any number of units of die area $n^2$, where n is a positive integer.
- Assume that any area not used by the large core will be filled with small cores.

- How large should Serena make the large core for the fastest possible execution of the program? (2p)
- What would the same program's speedup be if all 16 units of die area were used to build a homogeneous system with 16 small cores, the serial portion ran on one of the small cores, and the parallel portion ran on all 16 small cores?(1p)

# Answer Sheet for Task 3

| Inst. | ALUSrc1 | ALUSrc2 | ALUSrc3 | ALUOp1 | ALUOp2 | MemRead | MemWrite | RegWrite |
|---|---|---|---|---|---|---|---|---|
| **lw_add** | | | | | | | | |
| **addi_st** | | | | | | | | |
| **sll_add** | | | | | | | | |